



Московский
транспорт



Молоток

Кейс 05 · решение ReSense

Обнаружение посторонних объектов в тоннеле метро по данным 3D-лидара





О команде

- **Капитан:** Кузнецов Пётр, системный аналитик, ROS 2
- **Кол-во участников:** 4 человека
- **Краткое описание:**
 - P1 — узел ROS 2, Docker и питч; P2 — интерфейс
 - P3 — алгоритм, P4 — данные, метрики и тесты

КОМАНДА «Молоток»

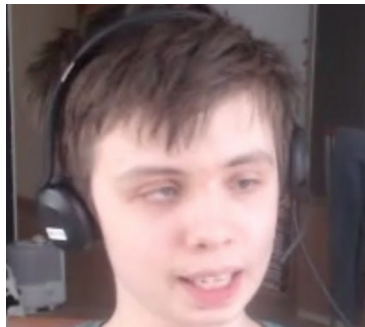
Краткое описание решения:

ROS 2-модуль: по потоку 3D-лидара строит модель «нормального тоннеля» — полотно, рельсы, ось пути с кривизной по стенам, — проверяет габарит поезда $2,1 \times 3,0$ м и отвечает GO / CAUTION / STOP / FAULT с расстоянием до препятствия. Решает геометрия; обученное «мнение» о треке лишь откладывает сомнительный STOP — не больше 10 обработанных кадров (~1 с при 10 Гц). Без GPU, в Docker.

Уникальность решения:

Описываем среду, а не объекты: крепление лидара и ось пути калибруются по рельсам в каждом кадре, кривизна — по стенам и рядам колонн, поэтому коридор осмыслен и там, где рельсов уже не видно. Публикуем оценку дальности контроля; возможны пропуски объектов. Настройка и оценка — на препятствиях, вставленных трассировкой лучей в реальные кадры.

КОМАНДА «Молоток»



**Кузнецов
Пётр**

P1 · капитан

- Системный аналитик, ROS 2
- требования, архитектура, узел ROS 2 и Docker, питч, сдача



**Максимов
Тимофей**

P2 · фронтенд

- Разработчик ПО (Python / JS)
- RViz, Foxglove, веб-дашборд, разметка, слайды, ролик



**Лихатин
Андрей**

P3 · зрение

- Инженер компьютерного зрения
- модель пути, габарит, кластеры, трекинг, дальность, скорость



**Голубев
Павел**

P4 · данные

- Специалист по данным
- датасет, синтетические препятствия, разметка, метрики, тесты, CI

Краткая история команды:

Четыре инженерных направления — ROS 2 и интеграция, визуализация, компьютерное зрение, данные и тесты — собраны вокруг задачи контроля габарита тоннеля по 3D-лидару.

01

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

В записях почти нет реальных препятствий, а станции и стрелки похожи на объекты. Поэтому система оценивает геометрию тоннеля и ищет препятствия в габарите; пропуски возможны и в пределах расчётной дальности контроля.

02

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

Препятствий в записях почти нет — «ставили» людей и ящики в реальные кадры трассировкой лучей лидара. Станции и стрелки давали ложные остановки — помогли ось по стенам, зона доверия и подтверждение 0,5 с. Чужие идеи — только через проверку: 8 идей из открытых решений других команд прогнали через полный гейт: 5 отклонили, 3 включили, одно потом выключили после своей проверки безопасности (оно прятало висящий кабель). Записи разные (два топики, 120° и 360°, наклон 3°) — узел сам находит вход и калибруется по рельсам.

03

Техническая суть решения

- ROS 2 Humble-узел на Python (numpy / scipy / scikit-learn) и необязательные ядра на C++
- PointCloud2 → модель пути → габарит 2,1 × 3,0 м → кластеры → решение; оба набора топик, автокалибровка крепления
- Результат: GO / CAUTION / STOP / FAULT, расстояние и /resense/status; подтверждение 0,5 с, «мнение» о сомнительных треках
- Кадр → решение через ROS, p95: 81–82 мс (360°) и 49–78 мс (120°) на 4-ядерной машине, 28.09; правила 29.09 — ещё ~2 мс; ядра на C++ сокращают время детектора на 38–57 %
- 13 759 реальных кадров; человек — 61 из 61 кадра, STOP без пропусков; ложные события — 2,1 на км в выборке (правила подбирались на этих записях)
- Без GPU: CPU-образ запускается без дополнительного ПО

Маркетинговая суть решения

Применение: контроль габарита на беспилотном и хозяйственном поезде

Дальше: скорость от одометрии, работа с мелкими объектами вдали, обучение на реальных препятствиях

Внедрение: docker run → ros2 bag play; адаптер торможения и его проверки — отдельный этап



Проблема

Поезд без машиниста должен видеть путь: 100 м — хорошо, 200 — очень хорошо, 300 — отлично. При 80 км/ч и замедлении 1–1,3 м/с²: 190–250 м без задержек. Тоннель почти всегда пуст: классов объектов для обучения нет.

Альтернативы

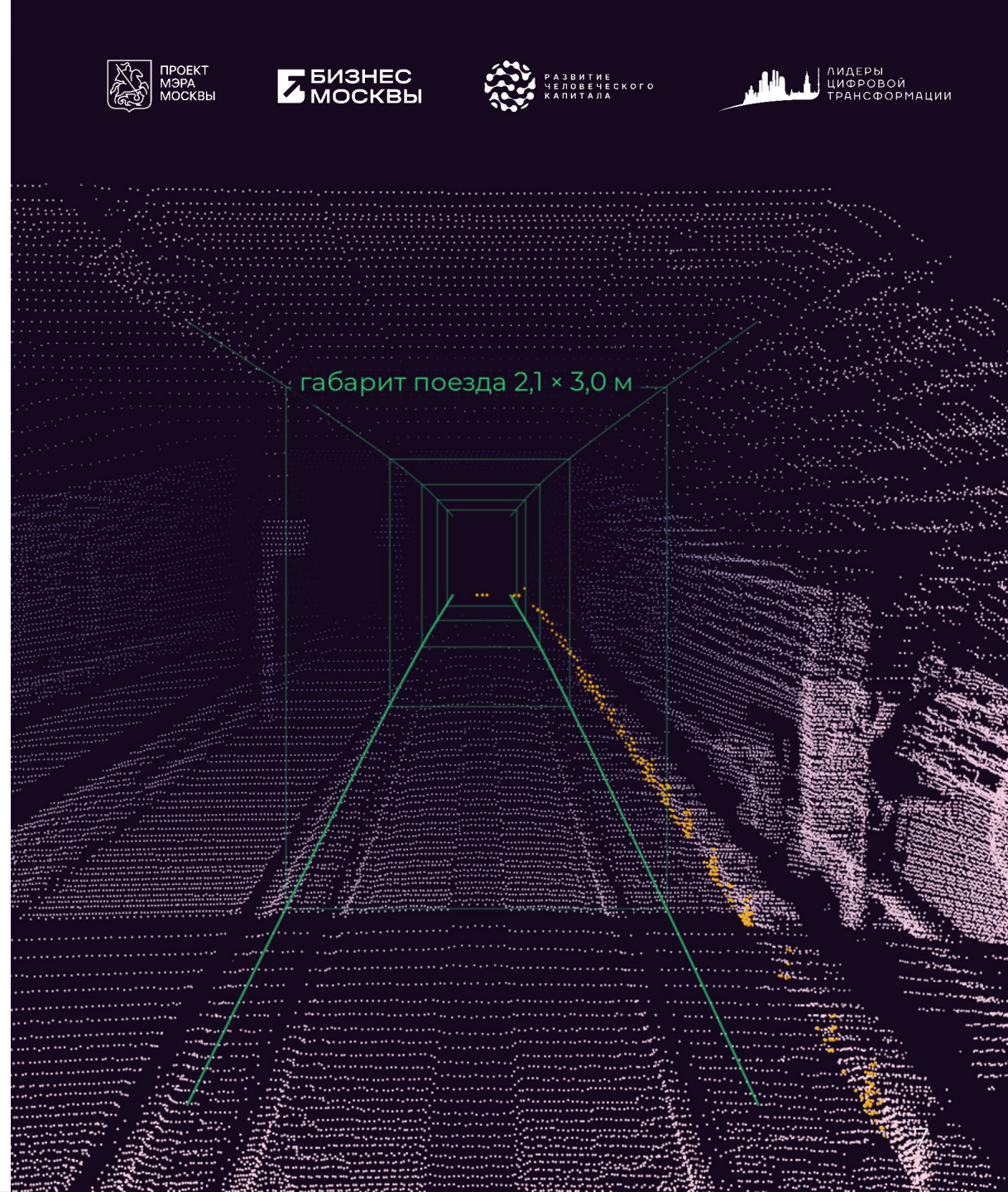
Обученный 3D-детектор требует размеченных препятствий; качество таких моделей на наших тоннелях не измерялось. Сравнение с картой требует локализации и повторного проезда.

Наше решение

Описать нормальный тоннель и искать препятствия в габарите поезда 2,1 × 3,0 м. Ось строится по рельсам, полотну и стенам без карты; обученное «мнение» может отложить сомнительный STOP. Есть пропуски и ложные тревоги.

ДАННЫЕ И СЕНСОР

- **13 759 кадров** — 6 записей (2 488) и 20-минутная поездка (11 271 кадр, 13 км, 7 остановок, кривые до $R \approx 350$ м)
- **Hesai Pandar128**, 10 Гц, ~190 тыс. точек в кадре: стены видны на 150–200 м, полотно — до ~100 м
- **~210 м — предел отражений в тоннеле:** каждая запись обрывается на 209–210 м, дальше нет ни одной точки (паспорт: 200 м при 10 % отражения)
- Объект 0,5 м даёт ~7 точек на 100 м и 1–2 точки на 200 м
- Реальные препятствия: человек и предмет на рельсе в записи doubleT_obstacle; всё остальное — синтетика в реальных кадрах, каждое число помечено



АЛГОРИТМ



Автокалибровка

вертикальная ось вращения;
ориентация и наклон — по
рельсам и полотну

1

Габарит

коридор 2,1 × 3,0 м вдоль оси и
ступень низких объектов на
рельсах

3

Решение

подтверждение 0,5 с, «мнение»
о треке, запрет эго-движения →
GO / CAUTION / STOP / FAULT и
дальность контроля

5

Модель пути

полотно, головки рельсов, ось;
кривизна по стенам и
колоннам до 150–200 м

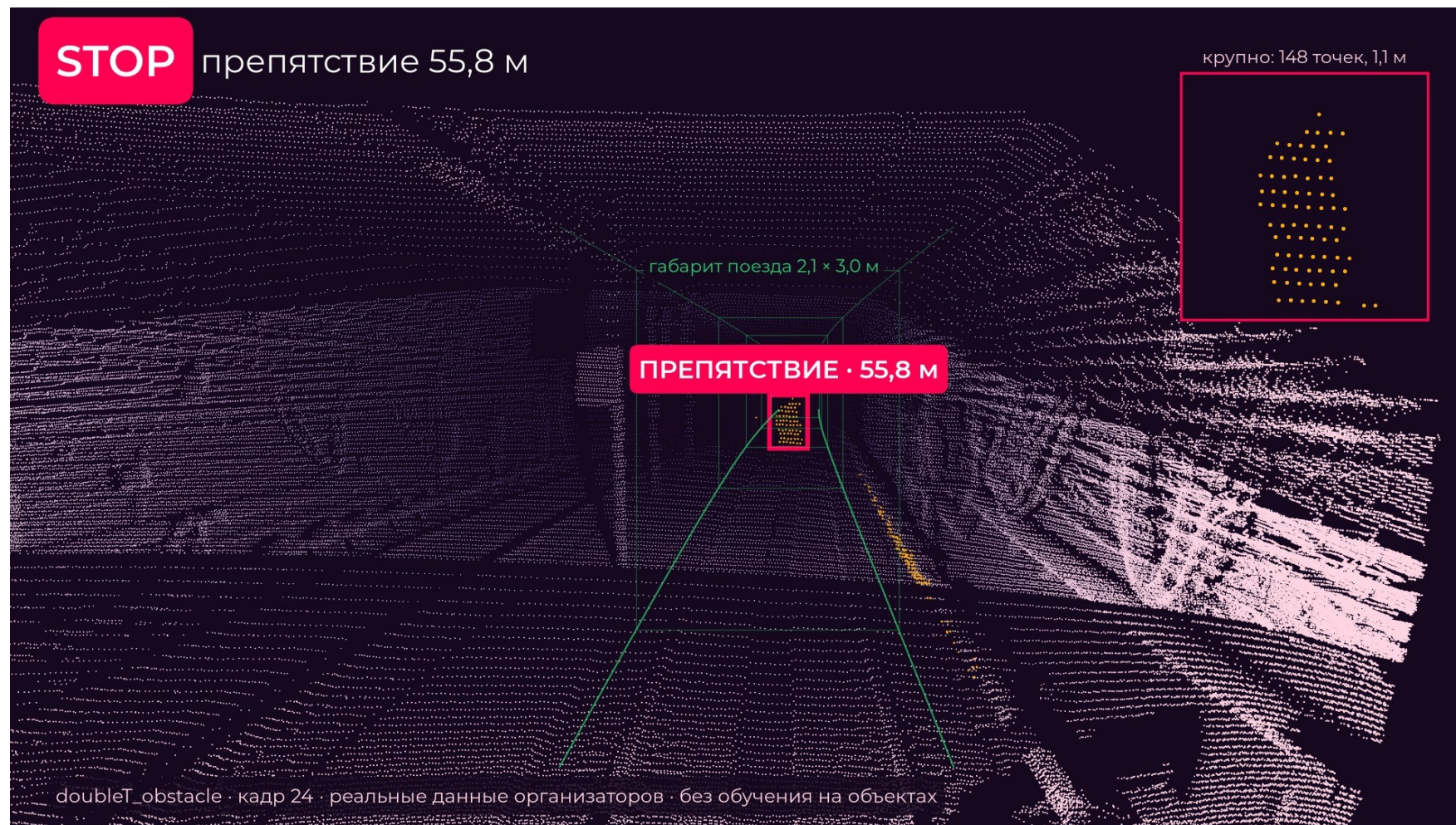
2

Кластеры

радиус растёт с дальностью;
фильтры инфраструктуры и
стен; дальнейшее правило

4

ГЛАВНЫЙ КАДР



Реальная запись
организаторов

Увидел за
56 м

человек переходит путь —
STOP

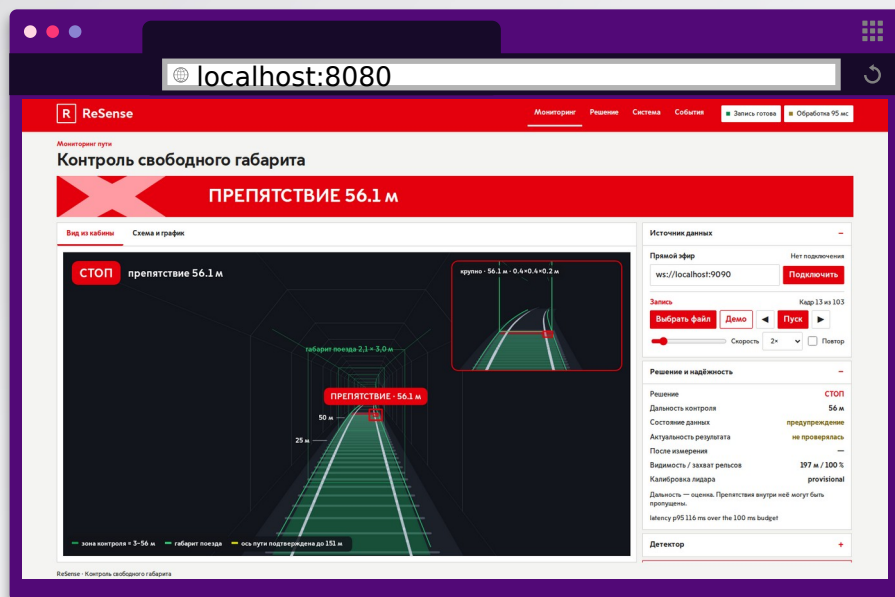
Найден в **61 из 61** кадра
внутри габарита (строго от
рельсов — 58)

Тревога с первого кадра в
габарите

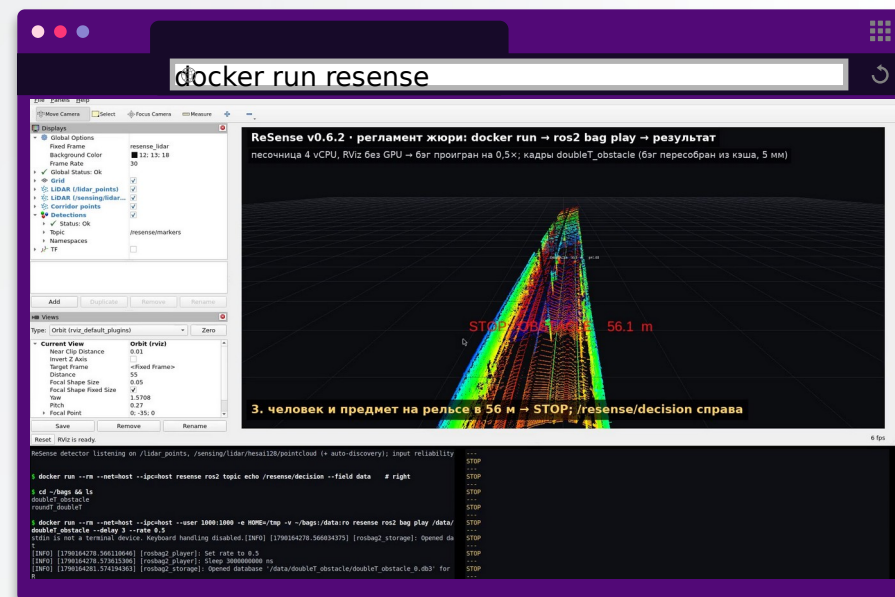
Ошибка дальности $\leq 0,23$ м

Вся запись: STOP на **193 из 201**
кадра — с кадра 8 до конца
без пропусков; предмет на
рельсе — 126 из 126

ДЕМОНСТРАЦИЯ



- Веб-дашборд «Контроль свободного габарита»: вид из кабины по статусу узла из Docker-прогона на doubleT_obstacle — STOP 56,1 м, оценка дальности контроля 56 м; живой узел — через rosbridge



- Цепочка жюри в Docker (архивная запись 23.09, 69 с — запасной показ): узел с RViz, bag play от обычного пользователя из другого контейнера, /resense/decision — STOP 56 м

docker load → docker run → ros2 bag play → /resense/decision

или одной командой: `scripts/play_bag.sh путь_к_бэгу [--archive resense-image-версия.tar.gz]`

13 759

реальных кадров прогнаны целиком

61 из 61

кадров с человеком на пути — тревога с первого кадра (строго от рельсов — 58: до 0,2 м по оси лидара ближе 60 м мы добавляем к габариту)

2,1 на км

ложных событий в поездке (27 за 13 км), в выборке — на ней же подбирались правила; вне выборки мерили только «мнение» (детектор 27.09): 2,8 на км

Реальная запись: человек — 61 из 61 кадра, ошибка $\leq 0,23$ м; STOP на 193 из 201 кадра — с кадра 8 до конца, без пропусков; предмет на рельсе — 126 из 126

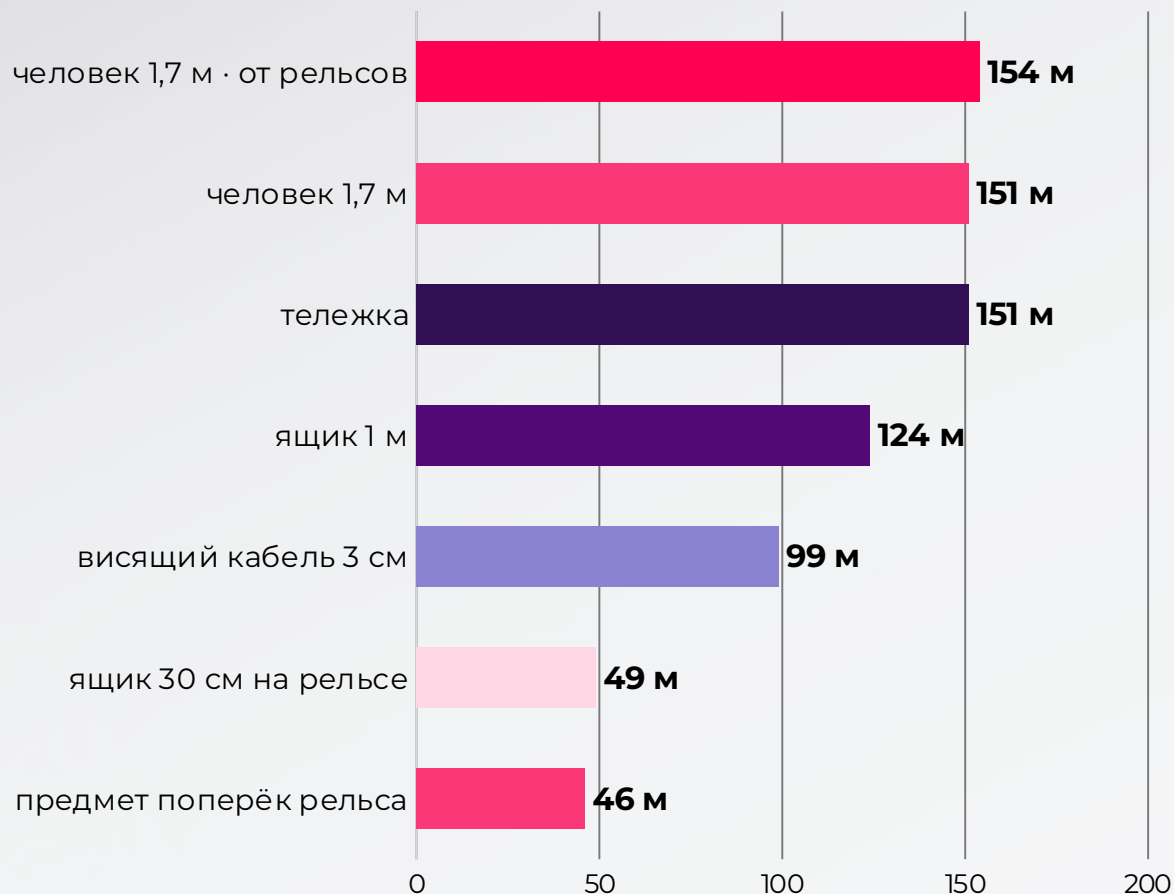
Ложные остановки (реальные, в выборке): поездка 13 км — 27 событий, 30 эпизодов STOP; пять пустых записей — 8 событий; до правил 29.09 из решений других команд — 32 и 11

Объекты организаторов [их синтетика]: STOP у 8 из 8 в габарите; ящик 2×2 м — с 98 м, доска — с 87 м, кубы 0,3 м — с 43–56 м, у края — с 35 и 18 м

Человек на подходе [наша синтетика в реальной поездке]: первое подтверждение — 151 м, медиана 6 подходов; ящик 1 м — 124 м, висящий кабель 3 см — 99 м; дальше ~210 м отражений нет

Задержка (p95, публикация кадра → решение, ROS в Docker): 360° — 81–82 мс, 120° — 49–78 мс на 4-ядерной машине (28.09); правила 29.09 — ещё ~2 мс; без GPU; стенд i7-9700E не измерен

ДАЛЬНОСТЬ



первое подтверждение на прямой, медиана 6 подходов (гейт 29.09, как и 27.09; ящик 30 см и предмет поперёк рельса — 24.09); «от рельсов» — объект поставлен по ближним рельсам, а не по дальней оси детектора (5 пар, прежняя постановка — 150 м); поезд 17–21 м/с без датчика скорости · синтетика в реальных кадрах поездки

• ~210 м — предел отражений в тоннеле

дальше 210 м во всех 13 759 кадрах нет ни одной точки

• Со скоростью поезда

накопление 5 кадров: человек 167 м (24.09); своя скорость по лидару (ошибка 0,07 м/с) раньше STOP не даёт

• Устойчиво

человек в $\geq 90\%$ кадров каждой полосы 10 м — со 115 м (24.09)

• В кривых

6 из 7 подходов, с 58–86 м — предел видимости за стеной ($R \approx 350$ м; 24.09)

ОБЪЕКТЫ ОРГАНИЗАТОРОВ



первый STOP, м; сиреневым — без устойчивого STOP · cloud_with_fake_obj: 10 объектов, 1 510 кадров, поезд едет к ним 1,4–20 м/с, скорость узлу не дана · синтетика организаторов

• STOP у 8 из 8, у края — с 18–35 м

ящик 2 × 2 м в центре — с 98 м, с первого появления; доска поперёк рельсов — с 87 м; кубы 0,3 м — с 43–56 м. На новых местах (72 случая, проверка заявлена заранее, детектор 27.09) — 723 из 2 458 кадров (было 544); на том же фоне без объекта — 0

• Мелкое вдали — поздно

на 60–115 м от куба 0,3 м 2–4 точки в кадре: дальше 60 м трек начинаем и с 4, но STOP — только если объект приближается, иначе CAUTION; ящик у верха габарита — STOP в 56 из 124 кадров — непрерывно с 111 м, дальше CAUTION или ничего

• Ограничения — говорим честно

габарит — от головки рельсов (ответ организаторов 29.09); ближе 60 м на прямой мы расширяем его до 0,2 м по оси лидара, но нигде не сужаем. Ложный STOP: 6 кадров у ящика снаружи (с 142 м), 0 вне объектов; синтетика расставлена приблизительно — организаторы это учтут

01

Калибровка

- 8 проверенных ориентаций вокруг вертикальной оси; крен и тангаж по рельсам и полотну; не любое крепление

02

Свежесть данных

- нет актуального статуса → FAULT или удержание STOP; потребитель проверяет /resense/status своим таймером

03

Любой вход

- оба набора топик / frame_id, поиск топика, перезапуск на новой записи

04

Воспроизводимо

- 1 279 тестов на путях nitrup и C++ (выход совпадает бит в бит); CI собирает образ и проигрывает бэги через узел; цепочка организаторов прогнана в Docker на реальных записях

Предметы на полотне. Наивный порог — 1 482 ложных события за 20 мин; после настройки остаётся слепое пятно: ящик 0,5 м найден в 2 из 6 подходов

Предмет на рельсе. Кластеризуем целиком: 2 → 126 из 126 кадров; один GO в кадре 111 закрыт 29.09 удержанием низкого STOP на 0,3 с — подобрано на этой же записи

Станции и стрелки. Ложные STOP у платформ; правила других команд — «чистая серия» и запрет эго-движения — 32 → 27 событий в поездке; их «обделку» мы выключили: висящий кабель получал STOP лишь с 21 м вместо 74 м

Мелкие объекты вдали. На 60–115 м всего 2–4 точки от куба; STOP для кубов 0,3 м — только с 43–56 м

У края габарита. STOP лишь с 35 и 18 м; висящий предмет 5 см — с 30 м (синтетика организаторов)

01

Что получилось

- ROS 2-модуль в Docker; все 13 759 реальных кадров: человек и предмет на рельсе — STOP без пропусков, ложные события — 2,1 на км (на данных, где подбирались правила); ящик организаторов — с 98 м; 1 279 тестов и CI; идеи других команд — только через полный гейт и свою проверку безопасности

02

Что дальше

- мелкие объекты раньше: короткие сигнатуры инфраструктуры, опора высоты по своду тоннеля; скорость от одометрии вместо оценки по лидару; «мнение» о треке — переобучить на реальных препятствиях

03

Внедрение

- `docker load → run → ros2 bag play` или одной командой: `scripts/play_bag.sh путь_к_бэгу [--archive resense-image-версия.tar.gz]` — образ, узел, запись и каждая смена решения с дистанцией; один файл параметров; стандартные сообщения ROS 2; адаптер к торможению ещё нужно разработать и проверить